DATA-260 Homework 8

DATA-260 Homework 8

GitHub repository: https://github.com/shrustishetty-2401/data260-9004

Student and Configuration

- Student: Shrusti Shetty
- SID4: 9004
- PORT_BASE: 8004
- PREFIX: `s9004`
- SEED: 9004
- VERIFY_SEED: 269004
- DOMAIN_ID: 4
- Hardware: MacBook Pro, Apple Silicon
- Local model: `qwen3:8b`
- Final branch: `hw8`
- Final tag: `hw8` Final tagged commit: `701d86691fb7f53273388130b1569b3d62da11d4`

Part 1: Kafka Vulnerability Request/Response Service

The primary domain entity is `Vulnerability`.

The worker stores entities in an in-memory dictionary. Clients communicate with the worker only through Kafka.

Kafka Topics

- `s9004_vulnerability_requests`
- `s9004_vulnerability_responses`

Request Schema

```
{
  "correlation_id": "unique-request-id",
  "reply_to": "s9004_vulnerability_responses",
  "operation": "CREATE_VULNERABILITY",
  "data": {
    "vulnerability_code": "HW8-DEMO-001",
    "vulnerability_title": "Kafka demonstration vulnerability",
    "package_name": "package-0",
    "submitter_email": "student@example.com",
    "description": "A sufficiently long vulnerability description.",
    "category": "Input Validation",
    "terms_accepted": true
  }
}
```

Response Schemas

Successful CREATE responses return the generated UUID:

```
{
  "correlation_id": "same-request-id",
  "success": true,
  "entity_id": "uuid",
  "message": "Entity created"
}
```

Successful GET responses return the complete entity:

```
{
  "correlation_id": "same-request-id",
  "success": true,
  "entity": {
    "entity_id": "generated-uuid",
    "vulnerability_code": "HW8-DEMO-001",
    "vulnerability_title": "Kafka demonstration vulnerability",
    "package_name": "package-0",
    "submitter_email": "student@example.com",
    "description": "A sufficiently long vulnerability description.",
    "category": "Input Validation",
    "terms_accepted": true
  }
}
```

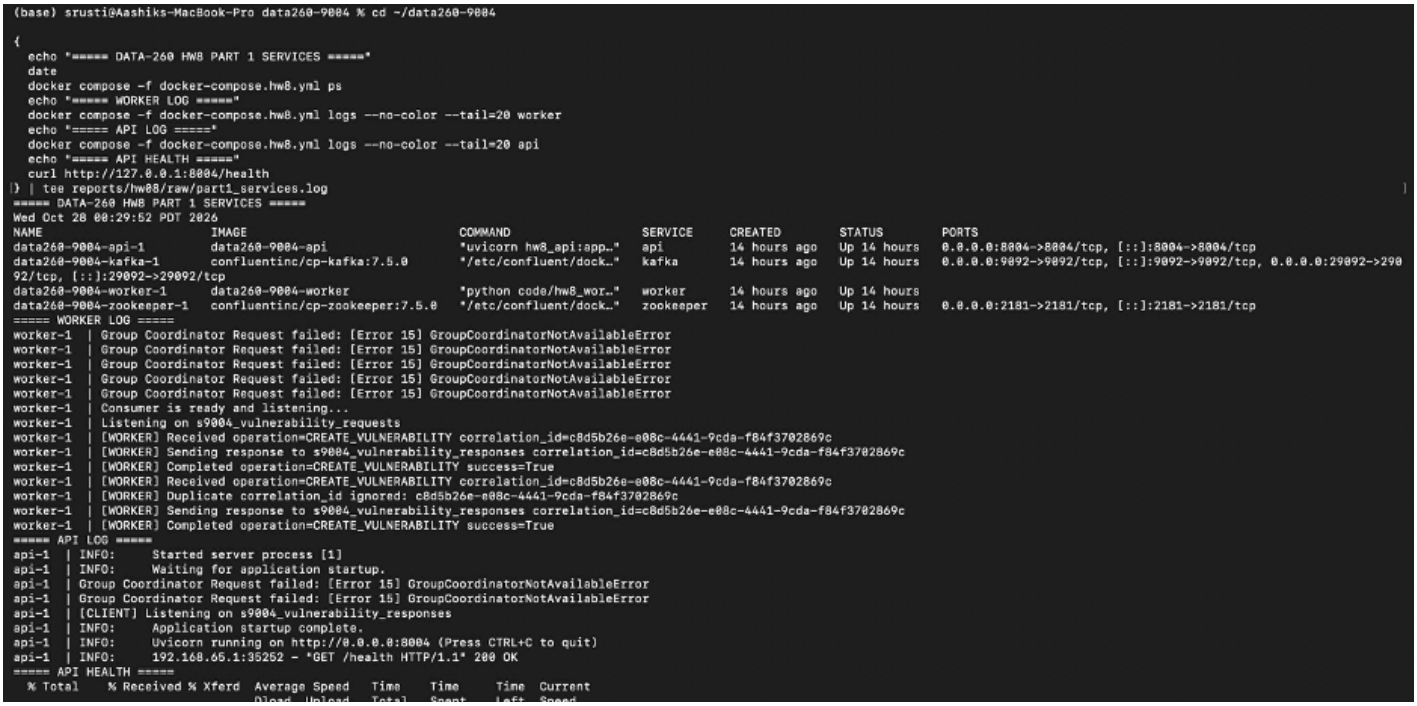
Error responses preserve the correlation ID and include an error message:

```
{
  "correlation_id": "same-request-id",
  "success": false,
  "error": "Entity not found"
}
```

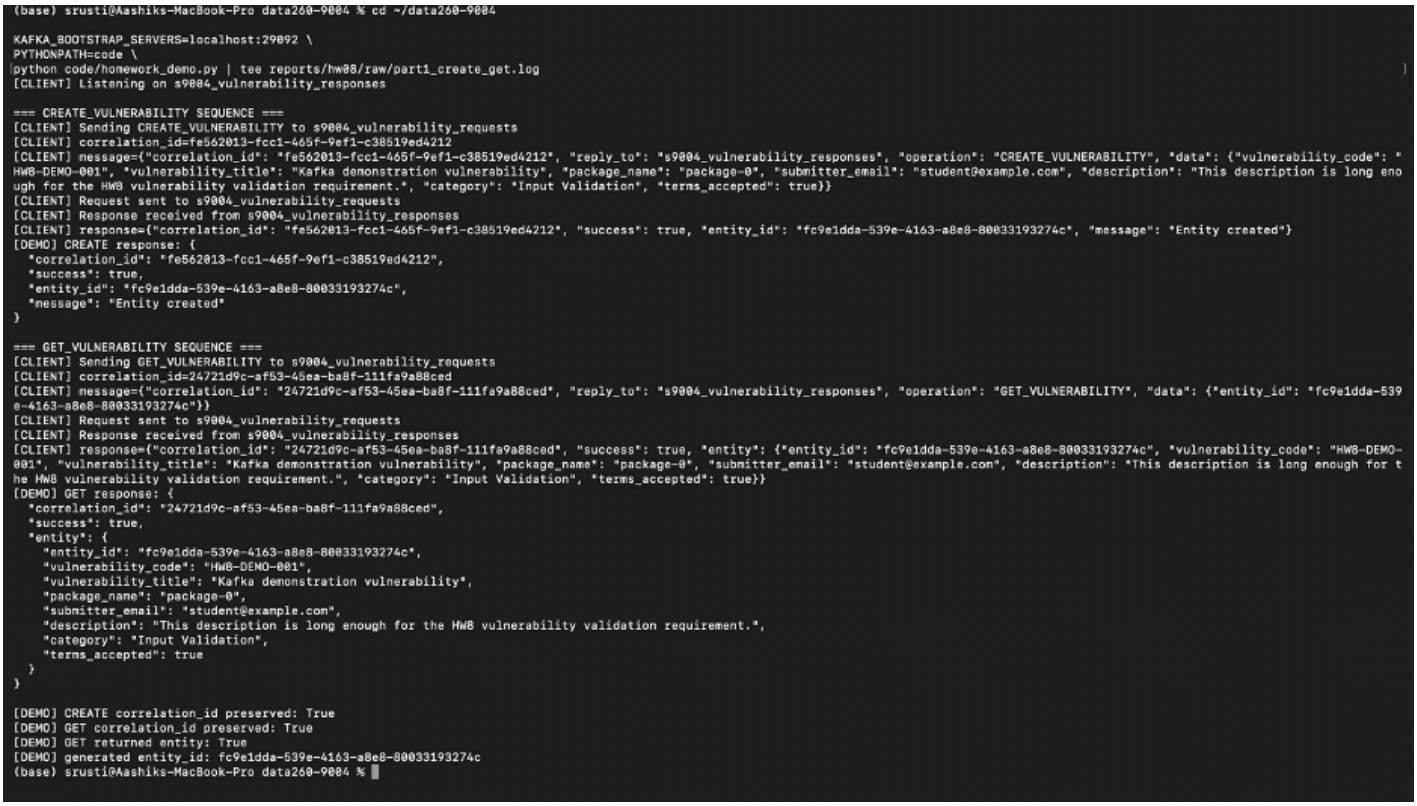

Functionality

- CREATE vulnerability requests
- GET vulnerability requests
- UUID entity identifiers
- Correlation IDs preserved in every response
- Input validation
- Unknown-operation errors
- Missing-entity errors
- Duplicate correlation handling

The CREATE then GET demonstration succeeded. The GET request returned the created vulnerability entity, and correlation IDs were preserved.



Part 1 service evidence



Part 1 CREATE and GET evidence

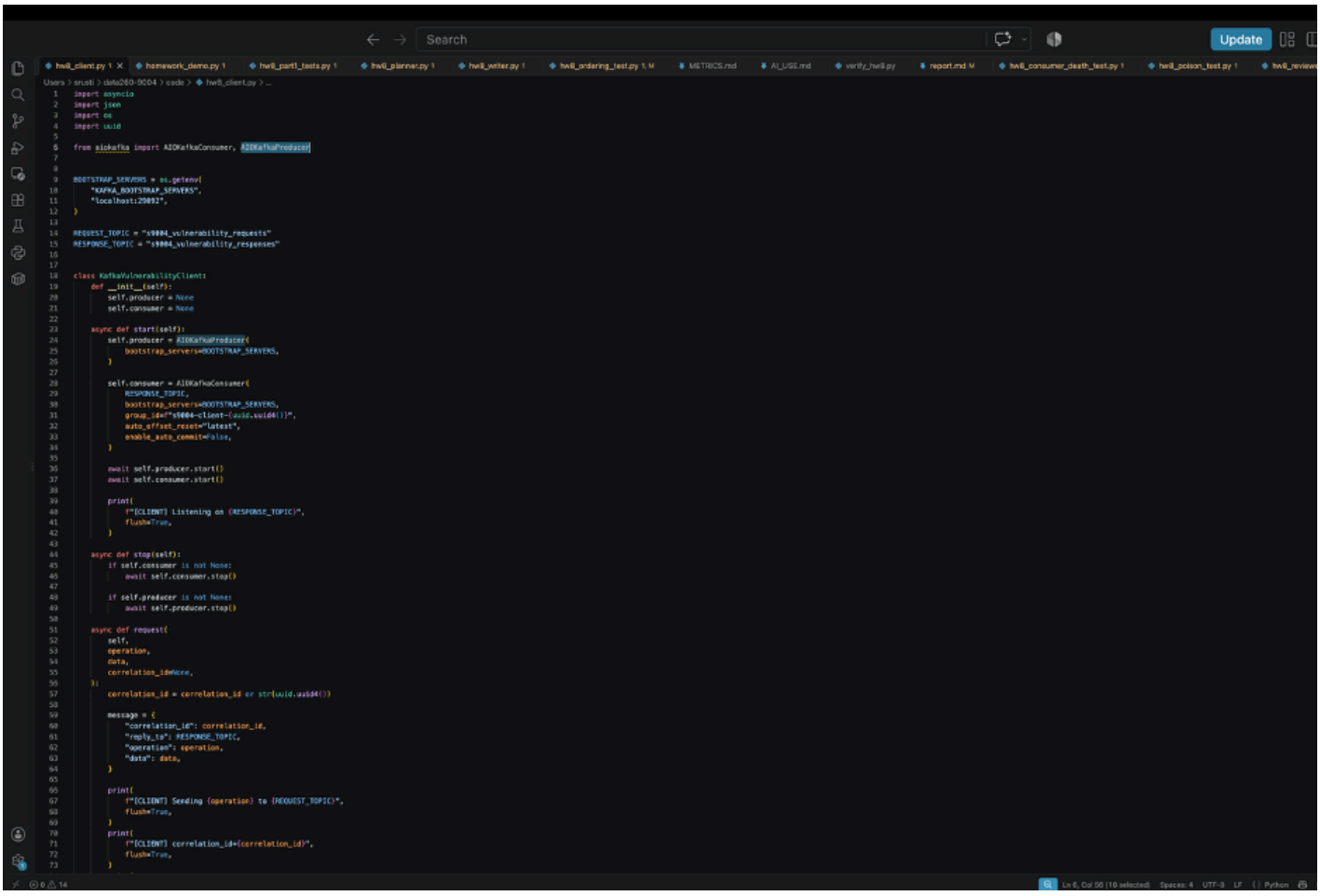
Evidence:

- `raw/part1_create_get.log`
- `raw/part1_error_tests.json`
- `raw/part1_services.log`
- `raw/part1_worker_full.log`
- `raw/part1_demo_full.log`

FastAPI Health Evidence

The API listens on `0.0.0.0:8004` inside Docker so that the container port is reachable. From the host, the service was verified through `http://127.0.0.1:8004/health` and returned HTTP 200.

Part 1 Implementation Evidence



```python
import asyncio
import json
import os
import uuid

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer


BOOTSTRAP_SERVERS = os.getenv(
    "KAFKA_BOOTSTRAP_SERVERS",
    "localhost:29092",
)

REQUEST_TOPIC = "s9004_vulnerability_requests"
RESPONSE_TOPIC = "s9004_vulnerability_responses"


class KafkaVulnerabilityClient:
    def __init__(self):
        self.producer = None
        self.consumer = None

    async def start(self):
        self.producer = AIOKafkaProducer(
            bootstrap_servers=BOOTSTRAP_SERVERS,
        )

        self.consumer = AIOKafkaConsumer(
            RESPONSE_TOPIC,
            bootstrap_servers=BOOTSTRAP_SERVERS,
            group_id="s9004-client-{uuid.uuid4()}",
            auto_offset_reset="latest",
            enable_auto_commit=False,
        )

        await self.producer.start()
        await self.consumer.start()

        print(
            f"[CLIENT] Listening on {RESPONSE_TOPIC}",
            flush=True,
        )

    async def stop(self):
        if self.consumer is not None:
            await self.consumer.stop()

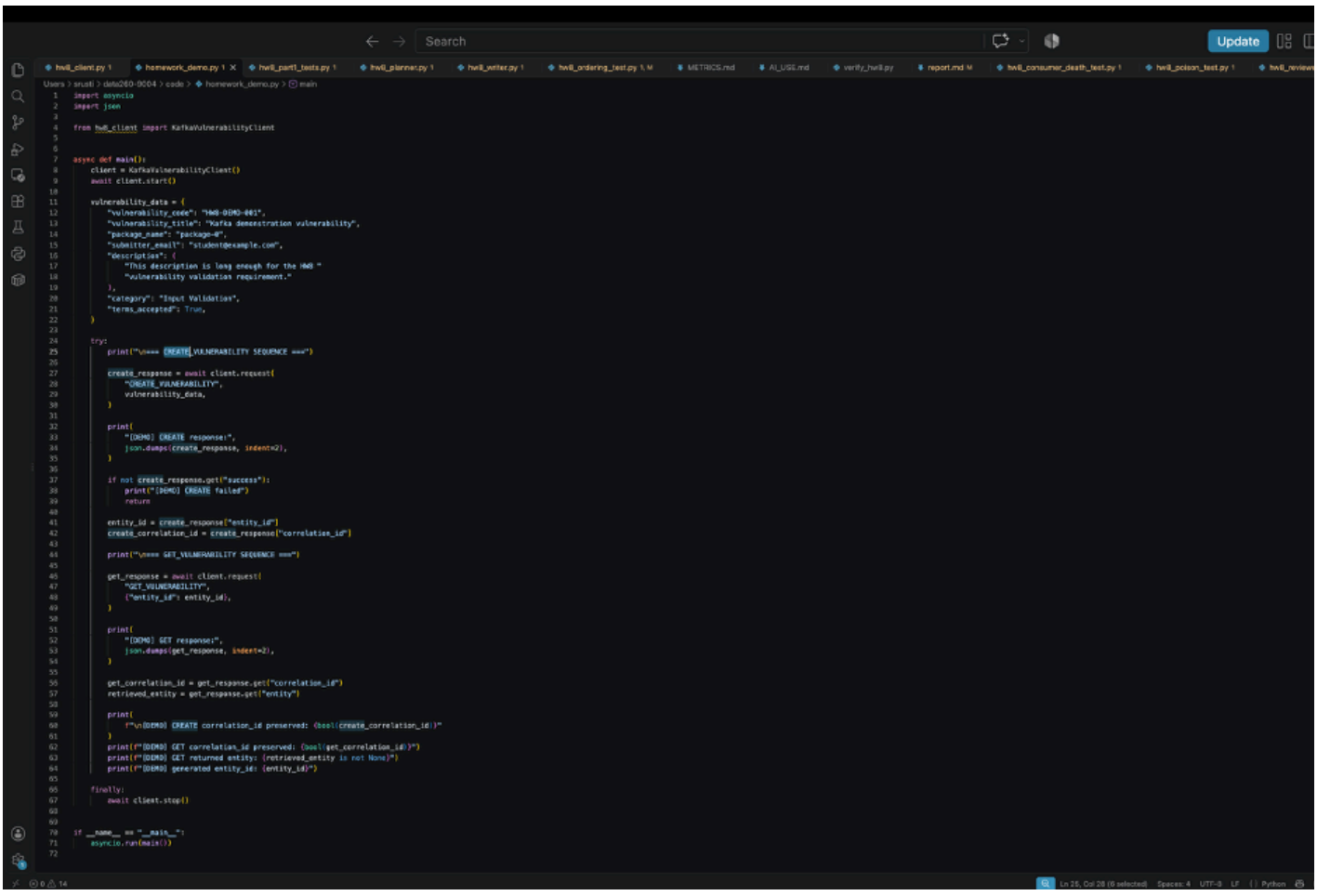
        if self.producer is not None:
            await self.producer.stop()

    async def request(
        self,
        operation,
        data,
        correlation_id=None,
    ):
        correlation_id = correlation_id or str(uuid.uuid4())

        message = {
            "correlation_id": correlation_id,
            "reply_to": RESPONSE_TOPIC,
            "operation": operation,
            "data": data,
        }

        print(
            f"[CLIENT] Sending {operation} to {REQUEST_TOPIC}",
            flush=True,
        )
        print(
            f"[CLIENT] correlation_id={correlation_id}",
            flush=True,
        )
```

Kafka client implementation



```python
import asyncio
import json

from hw8_client import KafkaVulnerabilityClient


async def main():
    client = KafkaVulnerabilityClient()
    await client.start()

    vulnerability_data = {
        "vulnerability_code": "HW8-DEMO-001",
        "vulnerability_title": "Kafka demonstration vulnerability",
        "package_name": "package-0",
        "submitter_email": "student@example.com",
        "description": (
            "This description is long enough for the HW8 "
            "vulnerability validation requirement."
        ),
        "category": "Input Validation",
        "terms_accepted": True,
    }

    try:
        print("\n=== CREATE_VULNERABILITY SEQUENCE ===")

        create_response = await client.request(
            "CREATE_VULNERABILITY",
            vulnerability_data,
        )

        print(
            "[DEMO] CREATE response:",
            json.dumps(create_response, indent=2),
        )

        if not create_response.get("success"):
            print("[DEMO] CREATE failed")
            return

        entity_id = create_response["entity_id"]
        create_correlation_id = create_response["correlation_id"]

        print("\n=== GET_VULNERABILITY SEQUENCE ===")

        get_response = await client.request(
            "GET_VULNERABILITY",
            {"entity_id": entity_id},
        )

        print(
            "[DEMO] GET response:",
            json.dumps(get_response, indent=2),
        )

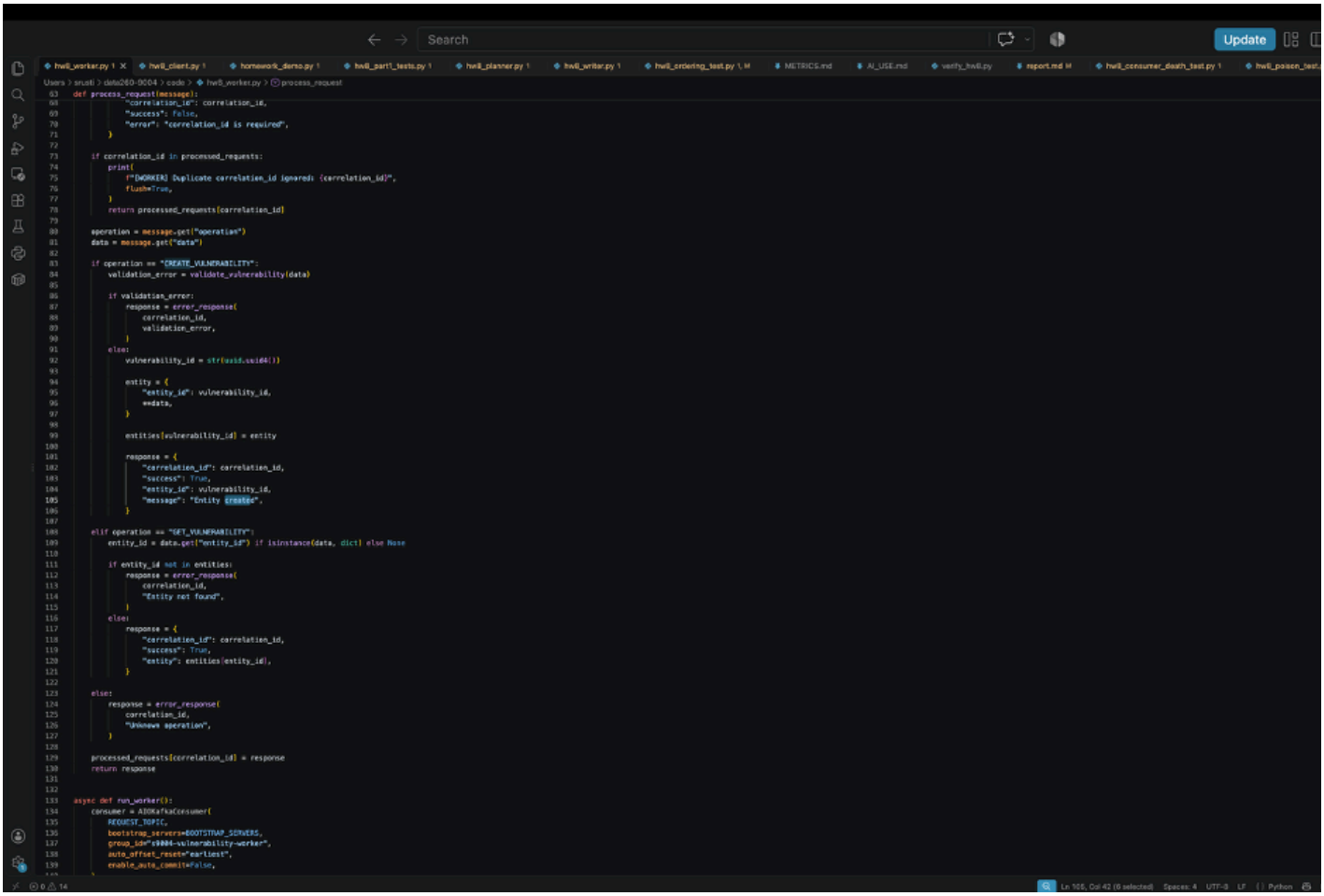
        get_correlation_id = get_response.get("correlation_id")
        retrieved_entity = get_response.get("entity")

        print(
            f"\n[DEMO] CREATE correlation_id preserved: {bool(create_correlation_id)}"
        )
        print(f"[DEMO] GET correlation_id preserved: {bool(get_correlation_id)}")
        print(f"[DEMO] GET returned entity: {retrieved_entity is not None}")
        print(f"[DEMO] generated entity_id: {entity_id}")

    finally:
        await client.stop()


if __name__ == "__main__":
    asyncio.run(main())
```

Homework demo implementation



```
def process_request(message):
            "correlation_id": correlation_id,
            "success": False,
            "error": "correlation_id is required",
        }

    if correlation_id in processed_requests:
        print(
            f"[WORKER] Duplicate correlation_id ignored: {correlation_id}",
            flush=True,
        )
        return processed_requests[correlation_id]

    operation = message.get("operation")
    data = message.get("data")

    if operation == "CREATE_VULNERABILITY":
        validation_error = validate_vulnerability(data)

        if validation_error:
            response = error_response(
                correlation_id,
                validation_error,
            )
        else:
            vulnerability_id = str(uuid.uuid4())

            entity = {
                "entity_id": vulnerability_id,
                **data,
            }

            entities[vulnerability_id] = entity

            response = {
                "correlation_id": correlation_id,
                "success": True,
                "entity_id": vulnerability_id,
                "message": "Entity created",
            }

    elif operation == "GET_VULNERABILITY":
        entity_id = data.get("entity_id") if isinstance(data, dict) else None

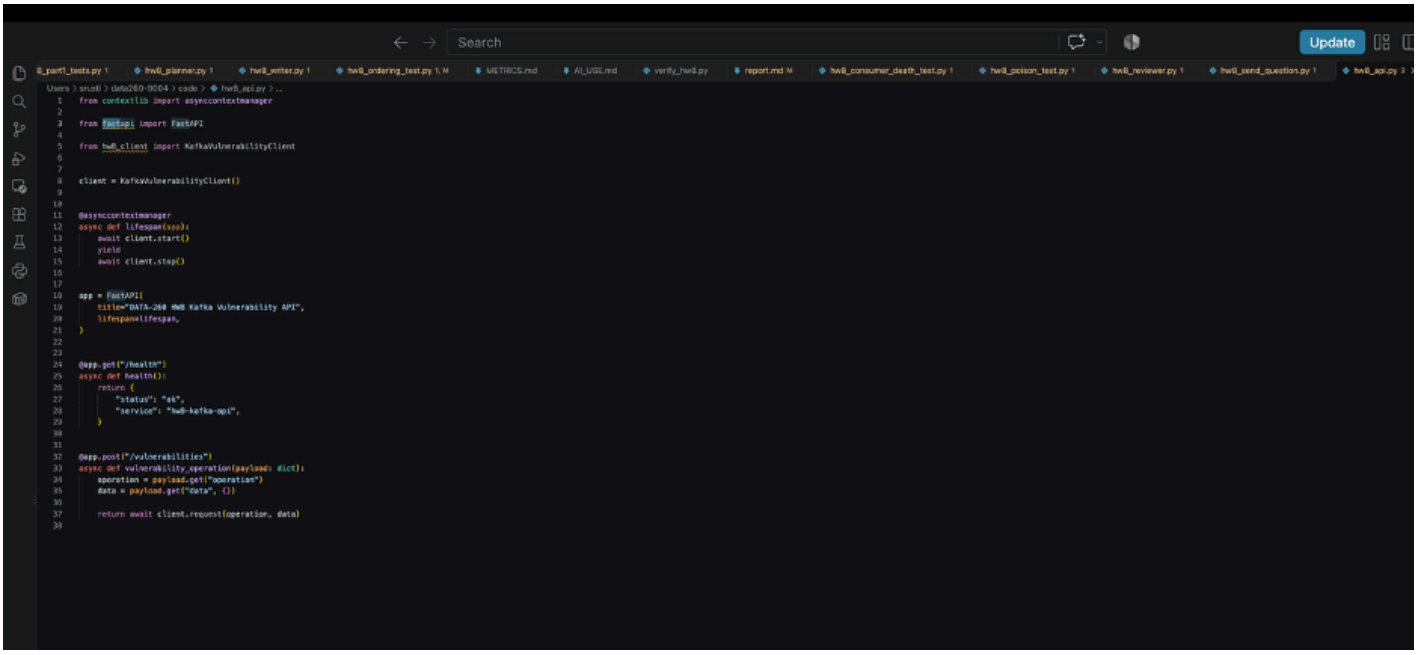
        if entity_id not in entities:
            response = error_response(
                correlation_id,
                "Entity not found",
            )
        else:
            response = {
                "correlation_id": correlation_id,
                "success": True,
                "entity": entities[entity_id],
            }

    else:
        response = error_response(
            correlation_id,
            "Unknown operation",
        )

    processed_requests[correlation_id] = response
    return response


async def run_worker():
    consumer = AIOKafkaConsumer(
        REQUEST_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id="s9004-vulnerability-worker",
        auto_offset_reset="earliest",
        enable_auto_commit=False,
```

Kafka worker implementation



```
from contextlib import asynccontextmanager

from fastapi import FastAPI

from hw8_client import KafkaVulnerabilityClient


client = KafkaVulnerabilityClient()


@asynccontextmanager
async def lifespan(app):
    await client.start()
    yield
    await client.stop()


app = FastAPI(
    title="DATA-260 HW8 Kafka Vulnerability API",
    lifespan=lifespan,
)


@app.get("/health")
async def health():
    return {
        "status": "ok",
        "service": "hw8-kafka-api",
    }


@app.post("/vulnerabilities")
async def vulnerability_operation(payload: dict):
    operation = payload.get("operation")
    data = payload.get("data", {})

    return await client.request(operation, data)
```

API wrapper implementation

```
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004

KAFKA_BOOTSTRAP_SERVERS=localhost:29092 \
PYTHONPATH=code \
python code/hw8_worker.py | tee reports/hw08/raw/part1_worker_full.log
Consumer is ready and listening...
Listening on s9004_vulnerability_requests
[WORKER] Received operation=CREATE_VULNERABILITY correlation_id=61488e8c-af58-4989-861b-7e0836f8b1f3
[WORKER] Sending response to s9004_vulnerability_responses correlation_id=61488e8c-af58-4989-861b-7e0836f8b1f3
[WORKER] Completed operation=CREATE_VULNERABILITY success=True
[WORKER] Received operation=GET_VULNERABILITY correlation_id=ae6a6715-3f25-4b6d-9755-d33ed0472d3c
[WORKER] Sending response to s9004_vulnerability_responses correlation_id=ae6a6715-3f25-4b6d-9755-d33ed0472d3c
[WORKER] Completed operation=GET_VULNERABILITY success=True
```

Complete worker message flow

```
Last login: Wed Oct 28 13:21:21 on ttys003
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004

KAFKA_BOOTSTRAP_SERVERS=localhost:29092 \
PYTHONPATH=code \
python code/homework_demo.py | tee reports/hw08/raw/part1_demo_full.log
[CLIENT] Listening on s9004_vulnerability_responses

=== CREATE_VULNERABILITY SEQUENCE ===
[CLIENT] Sending CREATE_VULNERABILITY to s9004_vulnerability_requests
[CLIENT] correlation_id=51488e8c-af58-4989-861b-7e0835f8b1f3
[CLIENT] message={"correlation_id": "51488e8c-af58-4989-861b-7e0835f8b1f3", "reply_to": "s9004_vulnerability_responses", "operation": "CREATE_VULNERABILITY", "data": {"vulnerability_code": "
HW8-DEMO-001", "vulnerability_title": "Kafka demonstration vulnerability", "package_name": "package-0", "submitter_email": "student@example.com", "description": "This description is long eno
ugh for the HW8 vulnerability validation requirement.", "category": "Input Validation", "terms_accepted": true}}
[CLIENT] Request sent to s9004_vulnerability_requests
[CLIENT] Response received from s9004_vulnerability_responses
[CLIENT] response={"correlation_id": "51488e8c-af58-4989-861b-7e0835f8b1f3", "success": true, "entity_id": "1fff659f-bf25-48a5-b90e-69846abb9089", "message": "Entity created"}
[DEMO] CREATE response: {
  "correlation_id": "51488e8c-af58-4989-861b-7e0835f8b1f3",
  "success": true,
  "entity_id": "1fff659f-bf25-48a5-b90e-69846abb9089",
  "message": "Entity created"
}

=== GET_VULNERABILITY SEQUENCE ===
[CLIENT] Sending GET_VULNERABILITY to s9004_vulnerability_requests
[CLIENT] correlation_id=ae5a6715-3f25-4b0d-9755-d33ed047263c
[CLIENT] message={"correlation_id": "ae5a6715-3f25-4b0d-9755-d33ed047263c", "reply_to": "s9004_vulnerability_responses", "operation": "GET_VULNERABILITY", "data": {"entity_id": "1fff659f-bf2
5-48a5-b90e-69846abb9089"}}
[CLIENT] Request sent to s9004_vulnerability_requests
[CLIENT] Response received from s9004_vulnerability_responses
[CLIENT] response={"correlation_id": "ae5a6715-3f25-4b0d-9755-d33ed047263c", "success": true, "entity": {"entity_id": "1fff659f-bf25-48a5-b90e-69846abb9089", "vulnerability_code": "HW8-DEMO-
001", "vulnerability_title": "Kafka demonstration vulnerability", "package_name": "package-0", "submitter_email": "student@example.com", "description": "This description is long enough for t
he HW8 vulnerability validation requirement.", "category": "Input Validation", "terms_accepted": true}}
[DEMO] GET response: {
  "correlation_id": "ae5a6715-3f25-4b0d-9755-d33ed047263c",
  "success": true,
  "entity": {
    "entity_id": "1fff659f-bf25-48a5-b90e-69846abb9089",
    "vulnerability_code": "HW8-DEMO-001",
    "vulnerability_title": "Kafka demonstration vulnerability",
    "package_name": "package-0",
    "submitter_email": "student@example.com",
    "description": "This description is long enough for the HW8 vulnerability validation requirement.",
    "category": "Input Validation",
    "terms_accepted": true
  }
}

[DEMO] CREATE correlation_id preserved: True
[DEMO] GET correlation_id preserved: True
[DEMO] GET returned entity: True
[DEMO] generated entity_id: 1fff659f-bf25-48a5-b90e-69846abb9089
(base) srusti@Aashiks-MacBook-Pro data260-9004 %
```

Complete client message flow



FastAPI health check

Part 2: Multi-Agent Kafka Pipeline

Implemented separate Planner, Writer, and Reviewer processes.

Kafka Topics

- `s9004_inbox`
- `s9004_tasks`
- `s9004_drafts`
- `s9004_final`

Agent Responsibilities

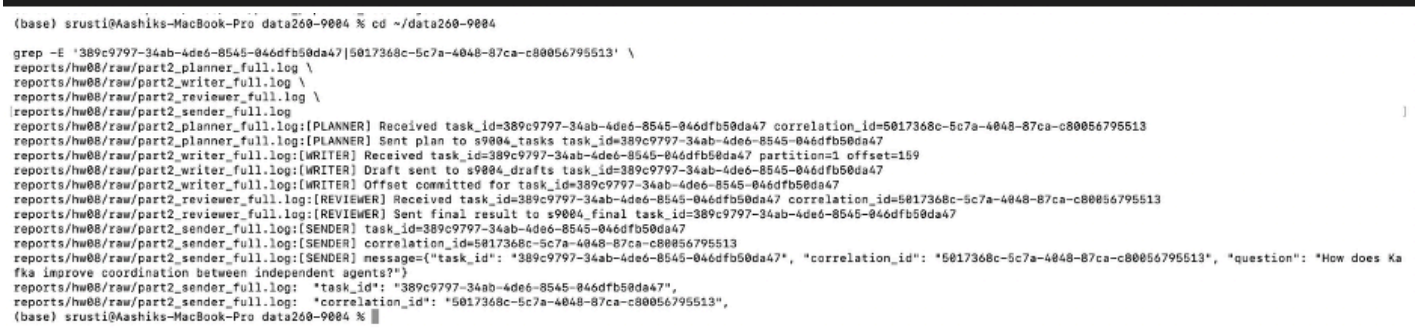
- Planner receives the question and creates a plan.
- Writer creates an answer draft.
- Reviewer evaluates the draft.
- Approved results are published to the final topic.

The pipeline completed successfully with status `approved`. The task ID and correlation ID were preserved through the pipeline.

Coordination Explanation

Kafka coordinates the agents through independent topics. The Planner publishes a plan to `s9004_tasks`, the Writer publishes a draft to `s9004_drafts`, and the Reviewer publishes the approved result to `s9004_final`. Task IDs and correlation IDs connect the stages without requiring the agents to run in one process.





Evidence:

- `raw/part2_pipeline_result.json`
- `raw/part2_pipeline.log`
- `RUN_LOG.txt`
- `raw/part2_planner_full.log`
- `raw/part2_writer_full.log`
- `raw/part2_reviewer_full.log`
- `raw/part2_sender_full.log`

For the same task, the Planner received the question and sent a plan to `s9004_tasks`; the Writer received that plan and sent a draft to `s9004_drafts`; and the Reviewer received the draft and sent the approved result to `s9004_final`. The task ID `389c9797-34ab-4de6-8545-046dfb50da47` and correlation ID `5017368c-5c7a-4048-87ca-c80056795513` were preserved across all stages.

Part 3: Reliability, Failure Handling, and Ordering

Duplicate Delivery and Idempotency

The same CREATE request was delivered twice. Both responses returned the same entity ID.

- `same_entity_id`: `true`
- `duplicate_prevented`: `true`

Evidence:

- `raw/duplicate_delivery.json`

Consumer Death and Recovery

Consumer Restart Evidence

The Writer was killed while processing task `b7bbc560-49e1-41f3-a711-158351720305`. After restart, Kafka redelivered the uncommitted task. The Writer completed the task, published the draft, committed the offset, and the test reported `recovered_after_restart: true`.

Evidence:

- `raw/part3_consumer_death_writer.log`
- `raw/part3_consumer_death_test.log`

The Writer was stopped during a simulated long-running task. After restarting the Writer, Kafka redelivered the uncommitted message and the task completed successfully.

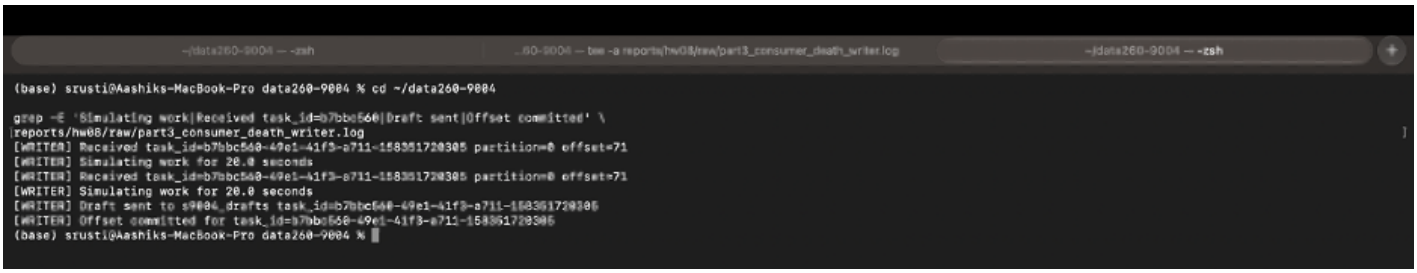
- `recovered_after_restart`: `true`
- `draft_received`: `true`

Evidence:

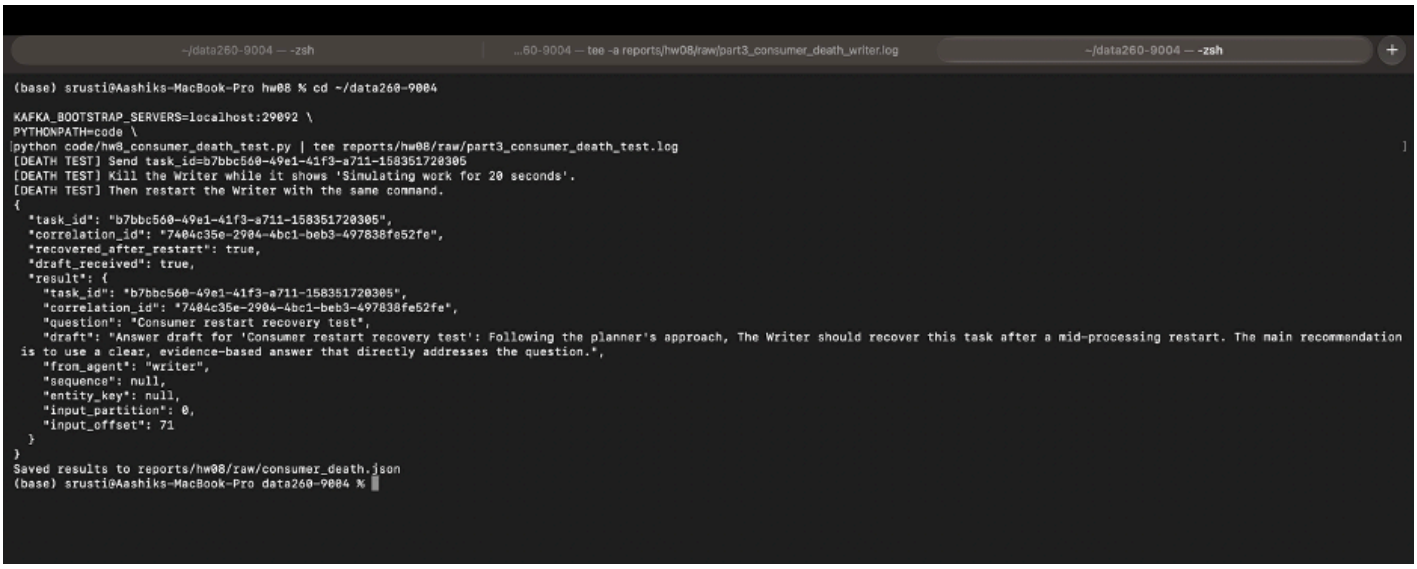
- `raw/consumer_death.json`

Evidence:

- `raw/consumer_death.json`



Consumer death Writer evidence



Consumer death recovery result

Poison Messages and Dead-Letter Handling

Poison messages intentionally failed processing. The Writer retried each message three times and then moved it to the dead-letter topic.

- Retry limit: 3
- Dead-letter topic: `s9004_dead_letter`
- `moved_to_dead_letter`: `true`

Evidence:

- `raw/poison_message.json`

Kafka Ordering

Twenty unkeyed and twenty keyed messages were tested.

Case	Partitions	Order preserved
Unkeyed messages	2	No
Keyed messages	1	Yes

The ordering experiment used two Kafka partitions and two Writer instances in the same consumer group. Unkeyed messages were distributed across partitions and were not globally ordered. Keyed messages used the same entity key, were routed to one partition, and preserved their order.

- `keyed_order_preserved`: `true`
- Verification seed: `269004`

```
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004

python - <<'PY' | tee reports/hw08/raw/part3_failure_summary.log
import json
from pathlib import Path

raw = Path("reports/hw08/raw")

duplicate = json.loads(
    (raw / "duplicate_delivery.json").read_text()
)
death = json.loads(
    (raw / "consumer_death.json").read_text()
)
poison = json.loads(
    (raw / "poison_message.json").read_text()
)
ordering = json.loads(
    (raw / "ordering_results.json").read_text()
)

print("===== DUPLICATE DELIVERY =====")
print("same_entity_id:", duplicate["same_entity_id"])
print("duplicate_prevented:", duplicate["duplicate_prevented"])

print("\n===== CONSUMER DEATH =====")
print("recovered_after_restart:", death["recovered_after_restart"])
print("draft_received:", death["draft_received"])

print("\n===== POISON MESSAGE =====")
print("retry_count:", poison["retry_count"])
print("dead_letter_topic:", poison["dead_letter_topic"])
print("moved_to_dead_letter:", poison["moved_to_dead_letter"])

print("\n===== ORDERING =====")
print(
    "unkeyed partitions:",
    ordering["unkeyed"]["partitions_seen"],
)
print(
    "unkeyed received order:",
    ordering["unkeyed"]["received_sequences"],
)
print(
    "keyed partitions:",
    ordering["keyed"]["partitions_seen"],
)
print(
    "keyed received order:",
    ordering["keyed"]["received_sequences"],
)
print(
    "keyed_order_preserved:",
    ordering["keyed_order_preserved"],
)
[PY
===== DUPLICATE DELIVERY =====
same_entity_id: True
duplicate_prevented: True

===== CONSUMER DEATH =====
recovered_after_restart: True
draft_received: True

===== POISON MESSAGE =====
retry_count: 3
dead_letter_topic: s9004_dead_letter
moved_to_dead_letter: True

===== ORDERING =====
unkeyed partitions: [0, 1]
unkeyed received order: [1, 2, 3, 7, 4, 5, 8, 10, 6, 9, 11, 12, 15, 16, 13, 14, 17, 18, 20, 19]
keyed partitions: [1]
keyed received order: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
keyed_order_preserved: True
(base) srusti@Aashiks-MacBook-Pro data260-9004 %
```

Part 3 failure and ordering evidence

Evidence:

- raw/ordering_results.json
- raw/part3_failure_summary.log
- raw/ordering_writer_a.log
- raw/ordering_writer_b.log
- raw/part3_writer_failure_full.log

- raw/part3_poison_full.log
- raw/part3_after_poison_full.log

```
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004

KAFKA_BOOTSTRAP_SERVERS=localhost:29092 \
PYTHONPATH=code \
python code/hw8_writer.py
[WRITER] Listening on s9004_tasks
```

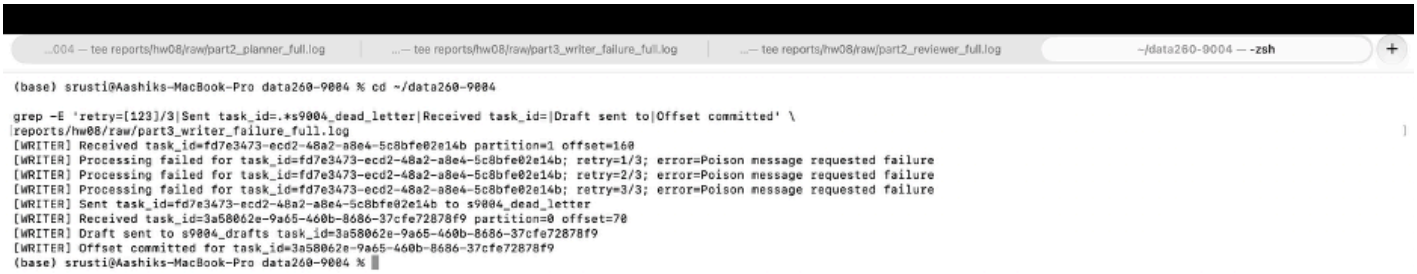
Writer A



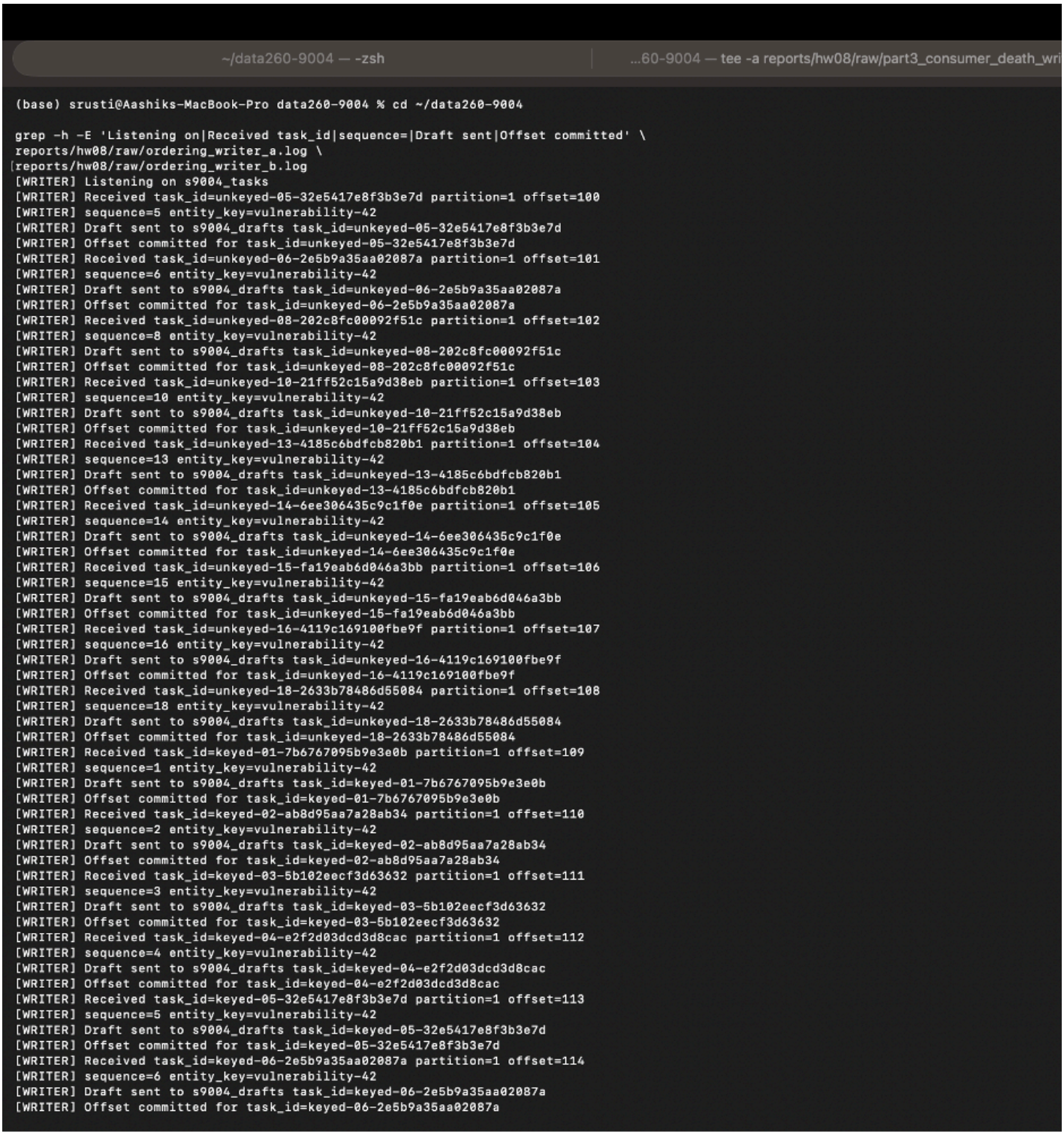
Writer B



Ordering result



Poison retry, dead-letter, and recovery



Writer A and Writer B processing evidence

Kafka Reliability Conclusions

Kafka provides:

- Durable message storage
- Consumer-group coordination
- Redelivery of uncommitted messages
- Retry and dead-letter processing
- Correlation between requests and responses
- Per-key ordering through partition assignment
- Decoupling between independent services

The producer is responsible for publishing valid messages and using stable keys when ordering matters. The consumer is responsible for validation, idempotency, retries, offset commits, and dead-letter handling.

The Writer was killed during a simulated long-running task before committing its offset. After restart, Kafka redelivered the uncommitted task. The Writer completed the task, published the draft, and committed the offset.

Comparison with HW2

HW2 used a simpler request/response workflow. HW8 extends that design with Kafka topics, independent consumers, consumer groups, offset commits, retries, dead-letter handling, idempotency, consumer restart recovery, and key-based ordering.

Kafka adds durable asynchronous coordination, but it also requires explicit handling of commits, retries, duplicate messages, consumer failures, and ordering.

Two benefits gained from Kafka were independent scaling of agents and decoupled failure recovery. Two new responsibilities were duplicate/idempotency handling and explicit ordering and offset management.

Failure Comparison Table

Failure scenario	Before fix	After fix
Duplicate delivery	Replay could create an additional side effect.	The same entity ID was returned and the duplicate was prevented.
Consumer killed mid-work	Processing stopped before the offset was committed.	Restart caused redelivery and successful completion.
Poison message	A bad message could continue failing without bounded handling.	The message was retried three times and moved to `s9004_dead_letter`.
Two partitions/two consumers	Unkeyed related messages were not globally ordered.	Keyed related messages used one partition and preserved order.

Each row is supported by the matching raw JSON or log evidence listed in the Part 3 sections above.

Short Implementation Evidence

The worker dispatches only the two required operations:

```python
if operation == "CREATE_VULNERABILITY":
    response = create_vulnerability(data)
elif operation == "GET_VULNERABILITY":
    response = get_vulnerability(data)
else:
    response = {"success": False, "error": "Unknown operation"}
```

Every response preserves the request correlation ID:

```python
response["correlation_id"] = message["correlation_id"]
await producer.send_and_wait(reply_to, json.dumps(response).encode())
```

Poison messages use bounded retries and then move to the dead-letter topic:

```python
if retry_number >= MAX_RETRIES:
    await producer.send_and_wait(DEAD_LETTER_TOPIC, encoded_message)
    await consumer.commit()
```The Writer retries failed messages in a bounded loop:

```python
while True:
    try:
        await process_message(message, producer)
        await consumer.commit()
        break
    except Exception as error:
        attempts[attempt_key] += 1
        retry_number = attempts[attempt_key]

        if retry_number >= MAX_RETRIES:
            await producer.send_and_wait(
                DEAD_LETTER_TOPIC,
                encoded_message,
            )
            await consumer.commit()
            break

        await asyncio.sleep(1)
```Ordering uses the related entity as the Kafka message key:

```python
key = b"vulnerability-42"
await producer.send_and_wait(TASKS_TOPIC, payload, key=key)
```

AI Use

1. AI assistance was used for drafting Kafka client, worker, multi-agent, retry, dead-letter, ordering, verification, and report code. The student ran the commands, inspected the outputs, captured the evidence, and made the final repository changes.
2. One initial AI-produced design created the Kafka producer outside an asynchronous function, which failed with a Python 3.13 event-loop error. Another initial retry design waited for Kafka redelivery instead of retrying poison messages immediately.

3. These problems were detected by running the Docker service and poison-message tests. The API failed during startup in the first case, and the poison test timed out in the second case.
4. The producer and consumer objects were moved into the asynchronous startup method. The Writer was changed to retry in a bounded loop, send the message to `s9004_dead_letter` after three failures, and commit the offset only after dead-letter publication.

Reproducibility Evidence

- `RUN_LOG.txt`
- `METRICS.md`
- `AI_USE.md`
- `verification.json`
- `raw/part1_create_get.log`
- `raw/part1_error_tests.json`
- `raw/part1_services.log`
- `raw/part2_pipeline_result.json`
- `raw/part2_pipeline.log`
- `raw/duplicate_delivery.json`
- `raw/consumer_death.json`
- `raw/poison_message.json`
- `raw/ordering_results.json`
- `raw/part3_failure_summary.log`
- `raw/part1_worker_full.log`
- `raw/part1_demo_full.log`
- `raw/part2_planner_full.log`
- `raw/part2_writer_full.log`
- `raw/part2_reviewer_full.log`
- `raw/part2_sender_full.log`
- `raw/part3_writer_failure_full.log`
- `raw/part3_poison_full.log`
- `raw/part3_after_poison_full.log`
- `raw/ordering_writer_a.log`
- `raw/ordering_writer_b.log`

Final Verification

The verification script reported:

- API health: passed
- Part 1 CREATE/GET evidence: passed
- Part 1 error tests: passed
- Part 2 pipeline approval: passed
- Duplicate delivery prevention: passed
- Consumer death recovery: passed
- Poison dead-letter handling: passed
- Keyed ordering preservation: passed
- Required evidence directory: passed

Overall result: `all_passed: true`

The final `verification.json` reported `all_passed: true` for API health, CREATE/GET evidence, error tests, pipeline approval, duplicate prevention, consumer recovery, dead-letter handling, keyed ordering, and the required evidence directory.

{ “homework”: “DATA-260 Homework 8”, “sid4”: 9004, “commit_hash”: “688891208660e40cb943af9927e0dd04f73ad9af”, “model”: “qwen3:8b”, “port_base”: 8004, “prefix”: “s9004”, “seed”: 9004, “verify_seed”: 269004, “domain_id”: 4, “checks”: { “api_health”: true, “part1_create_get_evidence”: true, “part1_error_tests”: true, “part2_pipeline_approved”: true, “duplicate_delivery_prevented”: true, “consumer_death_recovered”: true, “poison_dead_letter”: true, “keyed_ordering_preserved”: true, “required_evidence_directory”: true }, “all_passed”: true }

Verification JSON

The complete machine-readable verification output is stored in:

- `verification.json`

It records the homework name, SID4, commit hash, model, port configuration, seeds, domain ID, all verification checks, and the final `all_passed` result.

Submission Files

The final HW8 report is available here:

Open the final HW8 report PDF on GitHub

Local repository path:

`reports/hw08/Shetty_HW8.pdf`